

Automatic domain partitioning for quadrilateral meshing with line constraints

Nicolas Kowalski · Franck Ledoux · Pascal Frey

Received: 28 February 2014 / Accepted: 27 September 2014 / Published online: 9 December 2014
© Springer-Verlag London 2014

Abstract In this paper, we present an algorithm for partitioning any given 2D domain into regions suitable for quadrilateral meshing. It is able to preserve the symmetry of the domain if any, and can deal with inner boundaries and multidomain geometries. Moreover, this method keeps the number of singularities at the junctions of the regions to a minimum. Although each part of the domain, being four-sided, can be easily meshed using a structured method, we provide a meshing process that guarantees near perfect quality for most quadrilaterals of the resulting mesh. The partitioning stage is achieved by solving a PDE-constrained equation based on the geometric properties of the domain boundaries. An analysis of the generated mesh quality is provided at the end, showcasing that the meshes obtained through our algorithm are especially suitable for finite element methods.

Keywords Mesh generation · Finite element methods · partial differential equations · Quadrilateral · Partitioning · Automatic

1 Introduction

1.1 Problem statement

In numerous computational engineering applications, such as automobile crash simulations, structural mechanics, neutronics or fluid-structure interactions, quadrilateral meshes are preferred over triangular meshes [1]. Moreover, some numerical simulations can either require several materials, gas or fluids, leading to multidomain geometries, or possess internal line constraints that the mesh needs to abide by. The most desirable properties of quadrilateral meshes usually are:

1. to minimize the number of discrete singularities, defined here as the non four-valent vertices (internal vertices that do not have exactly four neighbors);
2. to align elements along the boundary, i.e., to have several layers of quadrilaterals aligned along the boundary in a parallel fashion;
3. to achieve high-quality elements, i.e., to get quadrilaterals as close to squares or rectangles as possible;
4. to respect a number of line constraints inside the surface that sometimes lead to multidomain geometries;
5. to be insensitive to orientation, i.e., to have the quality of the obtained mesh be independent from the orientation of the geometry
6. to have the elements abide by a prescribed size map.

Generating meshes fulfilling all those properties is a difficult task, especially since the optimization of one property may be in contradiction with the optimization of another one. Hence, a compromise between the various constraints and requirements is often needed. Several numerical and algorithmic solutions have been proposed over the last

N. Kowalski (✉) · F. Ledoux
CEA, DAM, DIF, 91297 Arpajon, France
e-mail: kowalski.nico@gmail.com

F. Ledoux
e-mail: franck.ledoux@cea.fr

N. Kowalski · P. Frey
UPMC Univ Paris 06, UMR 7598, Laboratoire J.-L. Lions,
75005 Paris, France
e-mail: frey@ann.jussieu.fr

decades to generate quadrilateral meshes abiding by those properties. Those methods can be categorized as either structured [2, 3] or unstructured [4–9], depending on the importance they give to the underlying structure of the mesh. On the one hand, structured methods are unable to mesh most geometries. On the other hand, unstructured methods usually yield meshes lacking some structure and having many undesired singularities. A good compromise between the geometrical flexibility of fully unstructured meshes and the numerical efficiency achieved on globally structured meshes can be achieved through block-structured meshes. Such meshes can be considered as unstructured arrangements of quadrilateral regions meshed in a structured way. The generation of such a mesh is a twofold process. At first, the domain has to be partitioned into quadrilateral parts that are overall much bigger than the desired mesh size. Then, each region has to be meshed into finer elements. The second step can easily be achieved using a transfinite interpolation [2]. However, the partitioning part is a tedious problem, especially for complex computational domains; for which no automatic solution is yet at hand, to the best of our knowledge. Such a partitioning can be done manually, but the time and expertise needed to do so is often prohibitive, even for industrial applications.

1.2 State of the art overview

Structured meshes are two-dimensional grids discretizing a domain, with no irregular nodes in their interior. While such meshes seem ideal in theory, most domains can not be meshed with a suitable grid. However, when the domain is appropriate, structured methods are both fast and efficient, leading to meshes having an excellent quality. Structured methods were initially designed using the transfinite interpolation [2], which provides a set of algebraic equations to mesh any quadrilateral domain using its boundary parametrization. More recently, the submapping technique was developed [3], allowing the meshing of a wider range of geometries. However, as this kind of mesh is specifically designed to introduce no internal singularity, most domains remain inappropriate for those techniques.

On the other hand, methods generating unstructured meshes are usually able to mesh any arbitrary geometry. Numerous methods fall within this category, with a large spectrum of variations developed over the years. Unstructured algorithms are often referred to as direct or indirect.

Direct methods aim at meshing the domain without relying on a background triangulation. In particular, advancing front methods [5] attempt to grow layers of quadrilaterals starting from the boundaries and filling the void space. Such approaches usually provide good element quality near the boundary. However, as the different fronts of elements merge or collide, areas that are difficult to mesh appear and can only be filled with elements of lesser quality and

numerous irregular vertices. A topological cleanup can then be performed to help alleviate this issue.

Indirect methods begin by triangulating the domain, before using the triangles to form quadrilaterals, by splitting them and pairing them [6, 7]. Lots of variations exist, trying to reduce the number of irregular vertices or to get better overall mesh quality, by moving the vertices of the background mesh [7] or creating a background mesh with right-angled triangles [8, 9], for example.

All those methods, however, do not consider the structure arising in a quadrilateral or hexahedral mesh, and as such produce meshes with a non-optimal number of discrete singularities, lowering the overall quality of the results, which can impact the accuracy and speed of the simulations using those meshes. It should also be noted that most of those methods fail to take into account internal line constraints and multidomain geometries. Several approaches were developed to deal specifically with those issues [10, 11], although the resulting mesh quality could still be improved.

Lastly, a few methods proceed by partitioning the domain into regions that are meshed afterwards. The use of the medial axis is especially useful to provide a decomposition of a domain [12]. However, the medial axis is numerically unstable, in that small changes of the domain can modify greatly the skeleton. Moreover, the resulting regions are generally not four-sided, and thus are not optimal for quadrilateral meshing. Even for a domain as simple as a square, a medial axis decomposition provides triangular parts, and not the grid one would expect. While this can be alleviated through the use of patterns, it would still be preferable to directly generate partitioning adapted to quadrilateral meshes.

The generation of quadrilateral meshes on 3D surfaces is a problem that possesses its own set of challenges. The two main differences between planar domain and 3D surfaces is that 3D surfaces may not be bound (a sphere, for example), and 3D surfaces possess principal directions of curvatures that can be used to drive the meshing process (and that usually need to be taken into account in the resulting meshes). Due to this close connection to principal directions of curvatures, both as a guide and as a constraint, methods developed for either planar domain or 3D surfaces may not be able to deal with the other type of domain. The full analysis of the state of the art regarding surface mesh generation goes beyond the scope of this paper; there exists, however, comprehensive surveys of the range of methods available [13].

1.3 Advantages of block-structured meshes over unstructured meshes

Block-structured meshes have one main advantage over other kind of meshes in that they can theoretically adapt to any given domain, like unstructured meshes, while maintaining the good element quality associated with structured

meshes. Moreover, block-structured meshes have two important added benefits that no unstructured mesh generation method would dispute.

The main feature of block-structured meshes is related to the logical rectangular structure of the individual regions, which are boundary-fitted non-orthogonal grids. From a numerical point of view, as a regular data structure is used for the representation of the field variables, efficient solvers can be employed within individual regions. Many numerical methods have been developed throughout the years to take advantage of such a structure [14–16]. From a computer science point of view, with the growing importance of high-performance computing, ratios of computational power to memory bandwidth have increased to the point that the maximum performance of current simulations is limited by the time for memory accesses [17]. Block-structured meshes have a hierarchical memory structure, allowing processors to have dedicated memories with quick access containing all of the information they need. Testing has shown the effectiveness of such a hardware configuration, for example, for iterative methods [18]. However, to the best of our knowledge, no algorithm has been developed to provide close to optimal partitioning of arbitrary domains into four-sided regions.

1.4 Overview of our approach

Given a 2D domain Ω of arbitrary shape and complexity, we introduce a new automatic domain partitioning method where the geometric features of $\partial\Omega$ are propagated in Ω by solving a PDE. By this way, we ensure the first three properties given previously to be satisfied. Our process starts with a uniform triangulation T_h of Ω . This triangulation is used to solve a PDE equation (see Fig. 1a and Sect. 2) that yields a directionality field (see Fig. 1b), to locate singularities of this field and to compute meaningful lines connecting those singularities. Using those, our approach achieves a partition of Ω into four-sided regions (see Fig. 1c and Sect. 3). This algebraic approach to domain partitioning can be used on any 2D domain, and can even deal with inner boundaries and multidomain geometries. It preserves the symmetry of Ω and minimizes the number of singularities. Although each region of the domain can then be easily meshed through a bilinear transfinite interpolation (see Fig. 1d), providing a good block-structured mesh, we also provide a meshing process based on the streamlines of the directionality field (see Sect. 5). This process guarantees near perfect quality for all quadrilaterals of the resulting mesh that are not connected to a singularity. While this second meshing process can be cost intensive when a high number of elements are required, the partitioning and the transfinite interpolation method are not, allowing the generation of good quality quadrilateral meshes with millions of

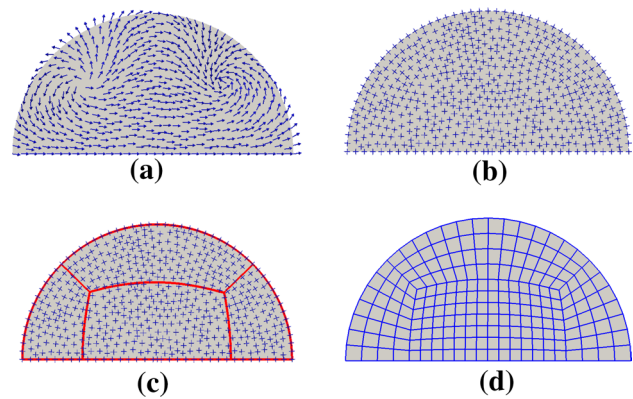


Fig. 1 Overview of the main steps of our algorithm. In **a**, the unit representation vector field obtained through the resolution of a PDE. In **b**, the corresponding cross field. In **c**, the domain partitioning in four quadrilateral regions. In **d**, a mesh of the domain obtained using a bilinear interpolation over each region (color figure online)

elements in a few seconds. Our method, however, can not create meshes with a prescribed varying size map. It creates meshes with constant size elements, thus lacking the fifth property a mesh should exhibit as outlined in Sect. 1.1. One way to achieve this would be to use our method to partition the domain and then to mesh each quadrilateral region using a method dealing with varying size maps.

Note that according to the classification of methods used previously, the meshing part of our approach belongs to the unstructured indirect category, as we are using a background triangular mesh and providing meshes that can contain discrete singularities. However, in contrast to most indirect methods, the background triangulation used has almost no impact on the quality of the obtained mesh.

1.5 Outline of the paper

This paper is organized as follows: in Sect. 2, we define the notion of directionality field and explain how to build such a field. In Sect. 3, we show how to partition the domain using streamlines of the directionality field previously defined. In Sect. 4, we provide solutions for several numerical issues. In Sect. 5 we describe how to obtain a quadrilateral mesh from this partitioning. Finally, in Sect. 6 we show and discuss meshes generated by our approach, so as to illustrate the features of our algorithm.

2 Creation of a directionality field

The main idea of this section consists of using a unit vector field prescribed on the vertices of ∂T_h , representative of its geometrical information, and propagating it inside T_h by solving a PDE. A first intuition would be to use the outer

normal vector field defined on the boundary. However, such a field is discontinuous at a corner with an angle of $\frac{\pi}{2}$, whereas the desired mesh at such a corner is perfectly regular. To overcome this problem, we use the concept of directionality [19], which is continuous on such corners and thus better fits the expected structure of quadrilateral meshes.

2.1 Definition of the directionality of Ω

A regular vertex P of a quadrilateral mesh of Ω is four-valent, and the four edges connected to P can be seen as two sets of opposite edges. Each of those sets, in turn, can be seen as the local discretization of a curve, with P being the intersection of those two curves. The two tangent directions of these curves at P can thus be described by a cross [19]. We consider the two directions of a cross to be orthogonal one to the other, which will help us to define quadrilaterals as close to squares or rectangles as possible, and we call this cross the directionality of the mesh at P . This notion of cross field is often used in computer visualization [19], and is usually defined as follows:

$$C_\theta = \left\{ \mathbf{v}_k = \left(\cos \left(\theta + \frac{k\pi}{2} \right), \sin \left(\theta + \frac{k\pi}{2} \right) \right)^T, 0 \leq k \leq 3 \right\}, \quad (1)$$

with $\theta \in [0, \frac{\pi}{2}[$.

2.2 One-to-one mapping between crosses and vectors

The first step of our approach consists in defining a smooth directionality field over T_h , save for a finite number of singular points where the field will not be defined, given its value at the vertices of ∂T_h . However, as a cross is a mathematical object difficult to handle, we can efficiently replace it by a corresponding representation vector [20] for our purpose. To this end, we introduce a one-to-one mapping between crosses and 2D unit vectors.

Given a cross C_θ defined by four vectors $\mathbf{v}_k$, $0 \leq k \leq 3$, each of them making an angle θ_k with an arbitrary fixed direction (for example, and in the following, the X axis),¹ we have

$$\theta_{k+1} = \left(\theta_k + \frac{\pi}{2} \right) [2\pi], \quad 0 \leq k \leq 2. \quad (2)$$

and we define the representation vector $\mathbf{v}$ of C_θ making an angle θ_r with the X axis such that :

$$\theta_r = (4\theta_k) [2\pi], \quad 0 \leq k \leq 3. \quad (3)$$

As a consequence, when a representation vector undergoes a 2π rotation, the corresponding cross only undergoes a $\frac{\pi}{2}$ rotation (see Fig. 2). This property allows us to have information that is continuous at the corner of a square, for example, in contrast to the information given by a metric.

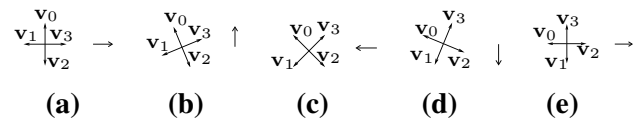


Fig. 2 Examples of crosses along the corresponding representation vectors. Notice that the crosses in **a** and **e** have the same representation vector, while the representation vector may be different from any vector of the cross

2.3 Definition of the representation vector field as the solution of a non-linear PDE

Our goal is to propagate information known on $\partial\Omega$ inside Ω in a smooth fashion. The solution of this problem can be considered as the steady state of a heat transfer problem. This consists here in minimizing a functional J defined as:

$$\begin{cases} J(\mathbf{u}) = \int_{\Omega} |\nabla \mathbf{u}|^2 dx, \\ \mathbf{u}(x) = \mathbf{u}_0(x) \quad \forall x \in \partial\Omega, \end{cases} \quad (4)$$

where x is a vector function. We add a non-linear constraint to keep vectors unitary. Indeed, allowing norms to vary introduce a bias leading to non-constant variations of the crosses orientations, which is not desirable as only directions are meaningful in our problem. Thus, given a geometrical domain Ω , we minimize the functional J defined as:

$$\begin{cases} J(\mathbf{u}) = \int_{\Omega} |\nabla \mathbf{u}|^2 dx, \\ \mathbf{u}(x) = \mathbf{u}_0(x) \quad \forall x \in \partial\Omega, \\ |\mathbf{u}(x)| = 1 \quad \forall x \in \Omega. \end{cases} \quad (5)$$

Here, $|| \cdot ||$ stands for the Frobenius norm, $|\nabla \mathbf{u}|^2 = \frac{\partial u_x^2}{\partial x} + \frac{\partial u_y^2}{\partial x} + \frac{\partial u_x^2}{\partial y} + \frac{\partial u_y^2}{\partial y}$ if $\mathbf{u} = (u_x, u_y)$. The Dirichlet condition is defined at a point P of $\partial\Omega$ as the representation vector corresponding to the cross defined by tangents and normals to $\partial\Omega$ at P . This guarantees that the corresponding cross field is locally "aligned" with each boundary in its vicinity.²

As such, this problem is ill-defined, as a solution to Eq. (5) may not exist. Indeed, the solution of the same equation without considering the non-linear constraint may result in some cases in a vector field having one or more zeroes; but as the non-linear constraint prevents the vector field from having any zero, the resulting vector field is not defined at some points. Instead of solving Eq. (5), we find an approximate solution defined on Ω deprived of a finite number of points by proceeding in two steps: first, we solve Eq. (5) while neglecting the non-linear constraint; then, we refine this approximation using a linearization of the norm constraint.

² The representation vector used at a C^0 corner of $\partial\Omega$ is the average of the representation vectors of the two geometrical edges connected to this corner.

¹ Note that results are independent of this choice.

2.4 Solving the linear approximation of the problem

By neglecting the non-linear constraint, Eq. (5) becomes a simple steady-state heat problem. We solve it by searching a weak solution (Eq. 6). The Lax–Milgram theorem guarantees the existence and the uniqueness of a solution to this problem.

$$\forall \mathbf{v} \in V, \int_{\Omega} \nabla \tilde{\mathbf{u}} \nabla \mathbf{v} dx = - \int_{\Omega} \nabla \mathbf{u}_0 \nabla \mathbf{v} dx \quad (6)$$

with $V = H_0^1(\Omega)$ being the space of H^1 functions vanishing on $\partial\Omega$, $\tilde{\mathbf{u}} \in V$ and $\mathbf{u} = \tilde{\mathbf{u}} + \mathbf{u}_0$.

We solve this equation numerically using a Galerkin finite element approach [21] on a triangulation T_h of Ω with P_1 -Lagrange elements.

2.5 Linearization of the norm constraint

The solution of the Dirichlet problem is a piecewise linear vector field defined on the triangulation T_h . More precisely, we obtained vector values at the vertices of T_h that could be used to define a continuous field using a piecewise linear interpolation scheme. Since we are interested in directions only, we have normalized the values of the field at the vertices of T_h . However, this increased the contributions to the value of the functional J , especially near the field singularities. To alleviate this problem, we reintroduce a posteriori the non-linear constraint $|\mathbf{u}| = 1$. Due to this non-linear constraint, the functional J is not convex, and, as it has a lower bound of zero, it means that it has several local energy minima. In our approach, we find the local minima closest to our first approximated solution using a linear approximation of the norm constraint [22] based on Lagrange multipliers.

The algorithm is based onto the fact that normalizing vectors with a norm greater than one reduces the value of the functional J . In practice, we use an iterative process starting with our approximation as initial solution. Each step of this process is divided into two parts, a PDE resolution and a normalization process, none of them adding to the value of the functional J . Let us describe the process between iteration n and $n + 1$. At each point i of the triangulation T_h , let $\mathbf{v}_{n,i}$ be the vector defined at point i during step n , we have $|\mathbf{v}_{n,i}| = 1$. To compute the solution at iteration step $n + 1$, we add a linear constraint C_i to this point:

$$\mathbf{v}_{n+1,i} \cdot \mathbf{v}_{n,i} = 1. \quad (7)$$

and we solve the previous steady-state heat problem under constraints C_i using the conjugate residuals method [23]. More precisely, to add the linear constraint at each point, we use Lagrange multipliers [24]. We thus obtain the following equation:

$$M \begin{pmatrix} u \\ \lambda \end{pmatrix} = \begin{pmatrix} A & C \\ C & 0 \end{pmatrix} \begin{pmatrix} u \\ \lambda \end{pmatrix} = \begin{pmatrix} b \\ 1 \end{pmatrix}, \quad (8)$$

with $\mathbf{1}$ the column matrix having each element equal to 1, λ the column matrix where each λ_i is the Lagrange multiplier associated to the constraint $|\mathbf{v}_i| = 1$, C a diagonal matrix, and, for $\mathbf{v}_{n,i} = (\alpha, \beta)$, $C(2i, 2i) = \alpha$ and $C(2i + 1, 2i + 1) = \beta$.

Eventually, we normalize all obtained vectors, which diminish the value of the functional J as the constraint guaranties us that $|\mathbf{v}_{n+1,i}| \geq 1$.

In practice, the system quickly converges to the closest local minima. As this process cannot add to the value of J , it is impossible to find another minima, even if it is lower than the closest one. This means that the topology of the vector field does not change during this operation. Instead, it allows the separatrices of the generated vector field (as defined in Sect. 3) to move slightly, and thus to have more regular curvatures. This increases the quality of the lowest quality elements in the resulting quadrilateral mesh, see Fig. 3.

3 Domain partitioning

At completion of the first step, we obtained a unit vector field defined at the vertices of T_h , which can be extended by interpolation to define a C^1 -continuous vector field over Ω (see Fig. 1a for an example of such a vector field). Each of these representation vectors corresponds to a cross, leading to a smooth cross field CF defined over all T_h (as depicted in Fig. 1b). The next step of our approach consists in analyzing the topology of this cross field. Finding the field singularities and computing the cross field streamlines that end on these singularities partition the geometry into parts that do not possess any field singularity in them, by definition (as is shown in Fig. 1c). The streamlines of the cross field thus possess a regular grid pattern inside each of these parts, which means it is trivial to fill it with a structured mesh (as is done in Fig. 1d). In addition to these field singularities, we consider the geometrical singularities of T_h ,

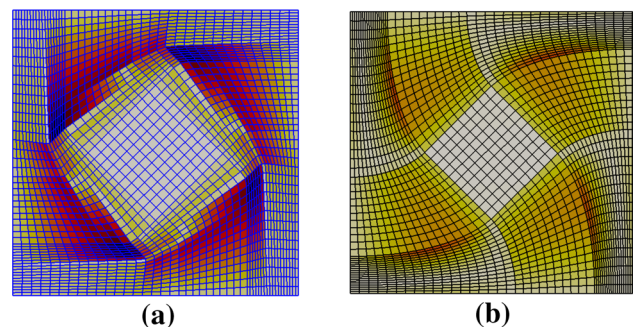


Fig. 3 Example of a square with a square hole or constraint inside it. In **a** and **b**, the obtained meshes, without and with the constraint, respectively, along with a minimum angle quality measurement (ranging from good, in white/yellow, to bad, in red/black) (color figure online)

which are the C^0 corners of the domain,³ so as to obtain a partition of T_h into regions that are not only devoid of any field singularities, but also quadrilateral.

The notion of topological skeleton of a vector field is well known [25–27], and can be understood as a set of curves representing the defining features of the vector flow (for example, the set of curves integrated from saddle points along the directions of the eigenvectors). We extend it now to cross fields, in a similar manner to what has been suggested in other works [19].

3.1 Singularities of a directionality field

3.1.1 Singularities detection

Definition 1 A singularity in a vector field $\mathbf{v}$ defined on T_h is a point x of T_h such that $\mathbf{v}(x) = 0$.

By extension, in our context, the singularities of the directionality field are detected as the zeroes of the piecewise linear interpolation of the representation vector field defined on the vertices of T_h . Moreover, we consider as singularities the discontinuities of the domain boundaries.

3.1.2 Singularity degree

We recall here some definitions and propositions [25, 26] related to the Poincaré index of the zeroes of a vector field.

Definition 2 The Poincaré index of an isolated zero P_0 of a 2D vector field $\mathbf{v} = (v_x, v_y)$ is the number of rotations of the vector field while traveling in positive direction along any closed curve enclosing P_0 but no other singularity. Put another way, for γ a closed curve containing P_0 and no other singularity, the Poincaré index i_{P_0} of P_0 is defined as:

$$i_{P_0} = \frac{1}{2\pi} \oint_{\gamma} d\phi, \quad \text{with} \quad \phi = \arctan \frac{v_y}{v_x}. \quad (9)$$

Proposition 3 Let $\mathbf{v}$ be a vector field on a 2D geometrical domain Ω . Suppose $\mathbf{v}$ is defined as a linear interpolation of non-zero values given at the vertices of a triangulation, T_h of Ω , all singularities are necessarily of first degree, i.e., their index is either 1 or -1 .

Proof A singularity S can either be inside a triangle or on an edge. If it is inside a triangle T defined by vertices P_1 , P_2 and P_3 , the vector field around S is defined only by the three

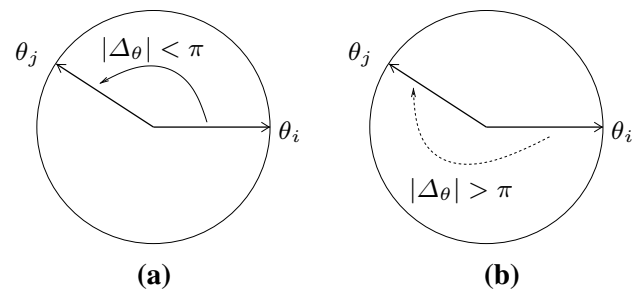


Fig. 4 Angular rotation along an edge. In **a**, a linear interpolation of the angle along the edge, and in **b**, a non-linear one. A linear interpolation always has an angle value smaller than π

vector values $\mathbf{v}(P_1)$, $\mathbf{v}(P_2)$ and $\mathbf{v}(P_3)$. We consider, without loss of generality, that the triangle $T = (P_1, P_2, P_3)$ is defined counter clockwise. For $i \in [1..3]$, let θ_i be the angle between $\mathbf{v}(P_i)$ and the X axis. For any couple $(i, j) \in [1..3]^2$, we define, $\Delta_{\theta_i^j} = (\theta_j - \theta_i + \pi)[2\pi] - \pi$, which is an angle defined on $] -\pi, \pi[$ that represents the angular rotation along the edge (P_i, P_j) when it is linearly interpolated from P_i to P_j (see Fig. 4). It is to be noted that if $\theta_i = -\theta_j$ then the angle cannot be linearly interpolated over the edge. $\square$

Thus, we have the equation:

$$\Delta_{\theta_i^j} = \int_{P_i}^{P_j} d\phi, \quad \text{with} \quad \phi = \arctan \frac{v_y}{v_x}. \quad (10)$$

The Poincaré index of S can now be expressed as:

$$i_S = \frac{\Delta_{\theta_1^2}}{2\pi} + \frac{\Delta_{\theta_2^3}}{2\pi} + \frac{\Delta_{\theta_3^1}}{2\pi}. \quad (11)$$

As each of the three angle differences along each edge is in $] -\pi, \pi[$, and as their sum is a multiple of 2π (since we integrate on a close loop along which the vector field is continuous), i_S is equal to either 0 or ± 1 . If it is 0, then the vector field is regular, and we do not have a singularity. Thus, i_S is equal to ± 1 .

If the singularity S is on an edge E between two triangles T_1 and T_2 , the vector field around S is described by the vector values on the vertices of T_1 , $\mathbf{v}(P_1)$, $\mathbf{v}(P_2)$ and $\mathbf{v}(P_3)$, and the vector values on the vertices of T_2 , $\mathbf{v}(P_1)$, $\mathbf{v}(P_2)$ and $\mathbf{v}(P_4)$ (we suppose, without loss of generality, that P_1 , P_3 and P_2 defines a travel along T_1 in a positive direction and that P_1 , P_2 and P_4 defines a travel along T_2 in a positive direction). Note that we only consider isolated singularities, which is the case with the vector fields generated using the process described in Sect. 2.⁴ This means that, using the same notations as above,

³ We consider a point of ∂T_h to be a corner if the two incident edges form an angle smaller than $\frac{3\pi}{4}$ or greater than $\frac{5\pi}{4}$.

⁴ A non-isolated singularity actually leads to an infinity of singularities which in turns leads to a much higher energy. As our resolution minimized it, it automatically removed such configurations.

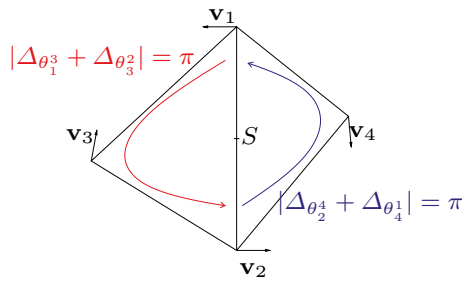


Fig. 5 Case of a singularity S along an edge. When integrating along the red path or along the blue path the angle difference is equal to π (color figure online)

θ_3 and θ_4 are each distinct to both θ_1 and θ_2 . Again, the Poincaré index of S can be expressed as:

$$i_S = \frac{\Delta_{\theta_1^3}}{2\pi} + \frac{\Delta_{\theta_3^2}}{2\pi} + \frac{\Delta_{\theta_2^4}}{2\pi} + \frac{\Delta_{\theta_4^1}}{2\pi}. \quad (12)$$

Using Eq. 13, we have, here again, i_S equal to either 0 or ± 1 ; and, as we do indeed have a singularity, we cannot have a null index, and therefore, i_S equals ± 1 (see Figure 5).

$$\frac{|\Delta_{\theta_1^3} + \Delta_{\theta_3^2}|}{2\pi} = \frac{1}{2}; \quad \frac{|\Delta_{\theta_2^4} + \Delta_{\theta_4^1}|}{2\pi} = \frac{1}{2}. \quad (13)$$

3.1.3 Minimum and maximum number of singularities

The index of a simple closed curve that encompasses several singularities is the sum of the respective indices of those singularities [25, 26]. It is then possible to define a minimum for the number of singularities of a given representation vector field.

Proposition 4 *Let us consider a domain Ω such that its outer boundary corresponds to a closed curve γ_0 of index i_0 and having n inner boundaries corresponding to curves γ_i , $i \in [1..n]$ of respective indices i_i , k singularities of index -1 and l singularities of index $+1$. Those indices are thus linked by*

$$i_0 = l - k + \sum_{i=1}^n i_i. \quad (14)$$

Consequently, the representation vector field has a minimum of $|i_0 - \sum_{i=1}^n i_i|$ singularities.

Proof This result is a direct consequence of considering only first-order singularities. $\square$

This result only depends on the value of the representation vector field on ∂T_h . Moreover, we are able to infer a maximum value for the number of singularities for the representation vector field resulting from the algorithm

described above. Unfortunately, we are unable to establish its existence formally. Nevertheless, this result proved to be accurate on each of our tests. To formulate this maximum, we first need to introduce a new notation for the variation of the representation vector field along a curve γ . Instead of considering a single index for this curve, we will use two real numbers, θ_1 and θ_2 , defined in Eqs. 15 and 16, respectively.

$$\theta_1(\gamma) = \frac{1}{2\pi} \int_{\gamma | (d\phi > 0)} d\phi, \quad \text{with} \quad \phi = \arctan \frac{v_y}{v_x}. \quad (15)$$

$$\theta_2(\gamma) = \frac{1}{2\pi} \int_{\gamma | (d\phi < 0)} d\phi, \quad \text{with} \quad \phi = \arctan \frac{v_y}{v_x}. \quad (16)$$

As we are working on T_h , with a field that is a piecewise linear interpolation of values defined at the vertices of T_h , we only consider curves γ_h that are closed non-intersecting successions of edges of T_h . On each edge e_i of γ_h , $d\phi$ is constant due to the piecewise linear nature of the field, allowing us to consider on e_i a constant $d\phi_{e_i}$. By denoting $|e_i|$ the length of the edge e_i , Eqs. 15 and 16 can thus be rewritten:

$$\theta_1(\gamma) = \frac{1}{2\pi} \sum_{e_i \in \gamma_h | (d\phi_{e_i} > 0)} d\phi_{e_i} |e_i|. \quad (17)$$

$$\theta_2(\gamma) = \frac{1}{2\pi} \sum_{e_i \in \gamma_h | (d\phi_{e_i} < 0)} d\phi_{e_i} |e_i|. \quad (18)$$

The information contained in the index of the curve is now split into two halves (see Eq. 19), one representing the positive variation of angles of the representation vector field, and one representing the negative one.

$$i_{\gamma_h} = \theta_1(\gamma_h) + \theta_2(\gamma_h) \quad \text{where} \quad \theta_1(\gamma_h) \geq 0 \quad \text{and} \quad \theta_2(\gamma_h) \leq 0. \quad (19)$$

Using these notations, the minimum number of singularities $S_{\min}$ of the representation vector field given in Proposition 4 can be redefined as follows:

$$S_{\min} = \left| \theta_1(\gamma_0) + \theta_2(\gamma_0) - \sum_{i=1}^n (\theta_1(\gamma_{i_h}) + \theta_2(\gamma_{i_h})) \right|. \quad (20)$$

Conjecture 5 *The maximum number of singularities $S_{\max}$ of the representation vector field can be expressed as:*

$$S_{\max} = |\theta_1(\gamma_0)| + |\theta_2(\gamma_0)| + \sum_{i=1}^n (|\theta_1(\gamma_{i_h})| + |\theta_2(\gamma_{i_h})|). \quad (21)$$

Two examples of those results are shown in Fig. 6. In Fig. 6a, d, curvatures contributing to $\theta_1(\gamma_0)$ are in green,

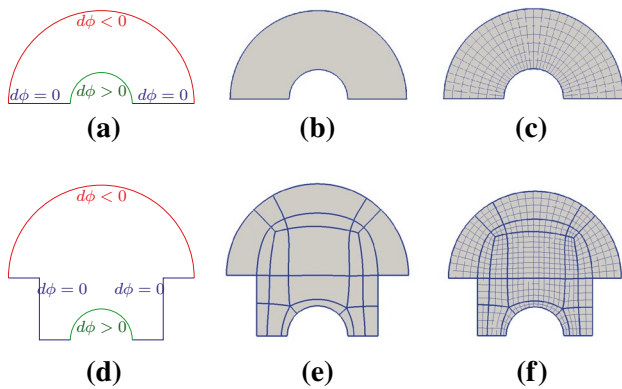


Fig. 6 Illustration of the minimum and maximum number of singularities on two examples; in **a**, **d**, curvatures contributing to $\theta_1(\gamma_0)$ are in green, curvatures contributing to $\theta_2(\gamma_0)$ are in red, and portions with null curvature are in blue. in **b**, **e**, the domain partitioning obtained through our algorithm is given, while **c**, **f** provides possible resulting meshes (color figure online)

curvatures contributing to $\theta_2(\gamma_0)$ are in red, and portions with null curvature are in blue. Figure 6b, e show the domain partitioning obtained through our algorithm in both cases. In addition, Fig. 6c, f illustrate meshes obtained by applying a transfinite interpolation in each region (see Sect. 5). In both examples, $\theta_1(\gamma_0) = \theta_2(\gamma_0) = 2$, leading to $S_{\min} = 0$ and $S_{\max} = 4$. In the upper example, our algorithm provides a solution where $S_{\min}$ is reached, while in the lower example, it provides a solution where $S_{\max}$ is reached. It is to be noted that diminishing the number of singularities in the lower example would imply distorting the representation vector field so that one of the three-valent singularities may overlap with one of the five-valent singularities. It would then lead to an extremely distorted representation vector field with a much higher functional value.

3.2 Streamlines and separatrices

3.2.1 Definition of a cross field streamline

A streamline of a vector field is a special kind of field line that can be defined as follows [25, 26]:

Definition 6 Let $\mathbf{v}$ be a continuous vector field defined over a domain Ω , and let $\gamma(s)$, $s \in]-\infty, +\infty[$ be the parametrization of a C^1 curve defined over Ω , with $\partial\gamma(s)$ its derivative. Then γ is a streamline of $\mathbf{v}$ if

$$\forall s \in]-\infty, +\infty[, \partial\gamma(s) \times \mathbf{v}(\gamma(s)) = 0, \quad (22)$$

where “ $\times$ ” denotes here the cross product of two vectors. A streamline is thus a C^1 -continuous curve for which the tangent to the curve at any given point has the same direction

as the vector field value at this point. It is to be noted that there is exactly one streamline going through any regular point of a vector field, and that if γ is not a closed curve then both end points of γ are either a singularity of the representation vector field or lie on $\partial\Omega$. Now we extend this concept to cross fields. Around any regular point of a smooth cross field, this field can locally be described as four smooth vector fields. It is only at singular points that such a separation of the components of the crosses is impossible. Thus, in regular areas (containing no singularity), this means that each regular point of a cross field can be described as the orthogonal intersection of two streamlines. Those streamlines can be computed, starting from this point, by selecting one of the four vectors defined at this point and, doing so, by selecting one of the four smooth vector fields and then using the same numerical scheme one would use to compute a vector streamline. By repeating this process, one can compute whole streamlines on the field as long as they do not go in the vicinity of field singularities. We can now give a proper definition for streamlines in cross fields.

Definition 7 Let $C = (\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \mathbf{v}_4)$ be a continuous cross field F defined over a domain Ω , save for a finite number of points that are the singularities of the field.⁵ Let $\gamma(s)$, $s \in]-\infty, +\infty[$ be a parametrization of a C^1 curve defined over Ω , with $\partial\gamma(s)$ its derivative. Then γ is a streamline of C if

$$\forall s \in]-\infty, +\infty[, \exists i \in [1..4], \partial\gamma(s) \times \mathbf{v}_i(\gamma(s)) = 0. \quad (23)$$

3.2.2 Creation of separatrices from singularities

For each singularity S , we now want to build separatrices, which are the streamlines of the cross field ending on S (the red streamlines in Fig. 7a, for example).

Definition 8 A streamline γ is a separatrix if there is a singularity S_0 such that $\exists t \in]-\infty, +\infty[, \gamma(t) = S_0$. In that case, we say that γ is a separatrix of S_0 .

To compute streamlines, we first need to define starting points around singularities, and then we need to build the streamlines until we either reach $\partial\Omega$ or another singularity. Should a streamline enter the vicinity of a singularity (in our case, the triangle of the background mesh T_h around S), we will consider it ends on it, thus allowing us to only compute streamline integration in the portion of the cross field that is well-defined.

⁵ We remind the reader here that this means that the representation vector field of F is continuous, and not that any of the $\mathbf{v}_i$ field is continuous.

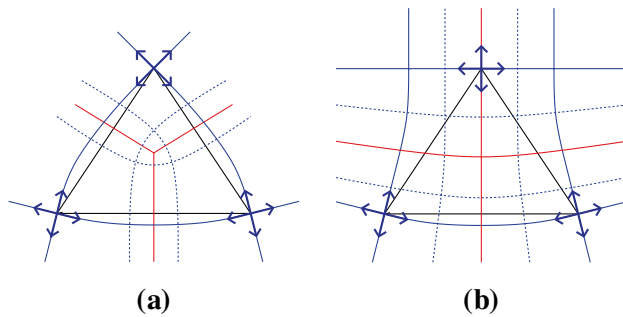


Fig. 7 In **a**, **b**, streamlines in a triangle containing a singularity or not (color figure online)

3.2.3 Relation between the number of separatrices of a singularity and its Poincaré index

The Poincaré index of a singularity of the representation vector field and the number of separatrices of the corresponding singularity in the cross field are closely related. Each singularity of the vector field having an index of $+1$ corresponds to a singularity of the cross field having three separatrices, and each singularity of the vector field having an index of -1 corresponds to a singularity of the cross field having five separatrices (regular points of the vector field correspond to points where a cross is defined and therefore are the intersection of two streamlines in the cross field).

Figure 8 depicts the shape of the cross field in each case, as well as the behavior of streamlines around a singularity. When we travel in positive direction along the edges of a triangle containing a singularity, we sum the linear angle difference (as defined in Fig. 4) of the representation vector over the three edges. This sum is 2π times the index of the singularity, according to its definition.

3.3 Skeleton and extended skeleton of the domain

The set of separatrices of the cross field along with their corresponding singularities define a pseudo-skeleton of the cross field. As we only consider one cross field over Ω , we call it the skeleton of Ω . While this skeleton does offer a partitioning of Ω , as all the separatrices ends either on a singularity or on the boundary of the geometry, there is no guarantee that the obtained regions are four-sided. In fact, it only provides regions that can be meshed through a sub-mapping technique [28], i.e., regions without any inner singularities. To get four-sided regions, we extend the skeleton by adding to the set of representation vector field singularities a set of geometrical singularities, which are defined as the C^0 corners on $\partial\Omega$. We then build separatrices for those geometrical singularities using the same process than for the representation vector field singularities.

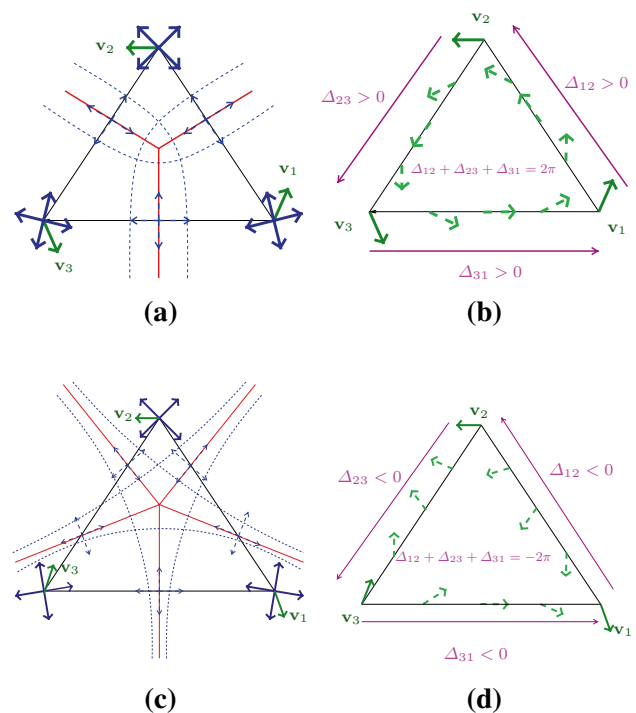


Fig. 8 A three-valent singularity, on top, and a five-valent singularity, on bottom. In blue, the local cross field, and in green the corresponding representation vector field. Separatrices are in red, along with the shape of surrounding streamlines in blue dots. On the right, in light green dots, the linear interpolation of the representation vectors along the edges is shown. The path along which we compute the index is the triangle around the singularity (color figure online)

Proposition 9 *The set of separatrices of both the geometrical singularities and the field singularities leads to a partition of Ω into four-sided, save for the regions containing a C^0 corner on $\partial\Omega$ that are triangular and can easily be splitted into three quadrilaterals.*

Proof By construction, all the regions we obtained contains no singularity except eventually on their corners. This means that a parametrization $f(u, v)$ of the inside of a region is always possible. Moreover, the boundaries of the considered region can always be included inside the parametrization by continuity, except if one edge of the region is of null length, which corresponds to a triangular region. In this case, one of the two sets of parallel streamlines of the region has all its streamlines that converge to a same point, which is the degenerated edge of the region (see Fig. 9). $\square$

For all the streamlines to behave like this, the angle at this corner of the region must be at maximum $\frac{\pi}{4}$, by continuity of the cross field inside the region. As the directions of the separatrices going from an inner singularity are uniformly distributed, due to the piecewise linear nature of the

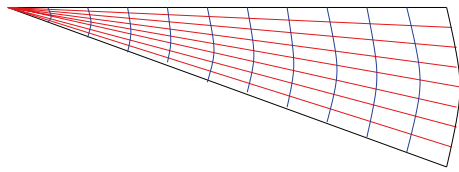


Fig. 9 A degenerated region where one of the boundary curve is of null length. This causes the region to be triangular instead of quadrilateral, and all streamlines in red to converge to the same point, preventing a proper parametrization of the region (color figure online)

cross field, and as we only have three and five-valent singularities, the minimum angle for a region corner that is a singularity inside Ω is $\frac{2\pi}{5}$. It follows that the degenerated edge of the considered region is a geometric corner featuring a small angle. In the absence of such a degenerated edge, the region can be parametrized by a one to one mapping function $f(u, v)$, with $u \in [0, 1]$ and $v \in [0, 1]$.

4 Numerical issues

In this section, we provide some details about technical issues arising in the method.

4.1 Computation of the separatrices location and direction on the triangle around a singularity

To compute the separatrices ending on a singularity S_0 , we start from S_0 and we build a streamline from there going into the direction with each of the separatrices ends on S_0 . Those directions need to be computed first. A straightforward way to do this is to compute the intersections between those separatrices and the triangle T_0 around S_0 , as well as the tangent of the separatrices at those points. Considering a point P on an edge of T_0 , a representation vector at P can be interpolated from the values on the vertices of the edge. This representation corresponds to a cross or to four vectors $\mathbf{v}_k$, $0 \leq k \leq 3$ representing the directions of the streamlines at P . A vector $\mathbf{u}$ can also be defined going from S_0 to P . So a streamline going through P is a separatrix of S_0 if

$$\exists k \in [0..3], \frac{\mathbf{v}_k}{|\mathbf{v}_k|} = \frac{\mathbf{u}}{|\mathbf{u}|}. \quad (24)$$

In this case, the tangent of the streamline at P is $\mathbf{v}_k$ (see Fig. 10).

As the location of P and the value of its cross can be linearly interpolated from the ones of the end points of the considered edge, it is straightforward to define all the separatrices going through this edge and, by repeating the process over the edges of T_0 , to define all the intersections between separatrices of S_0 and T_0 . Once this is done, the separatrices are approximated using a Runge-Kutta

numerical scheme described below until either the boundary of the geometry or another triangle containing a singularity is met.

4.2 Directionality interpolation over a triangle with no singularity

Considering a triangle $T \in T_h$, a point $X_i \in \partial T$ and a direction $\mathbf{d}_i$ in X_i , our aim is to compute an output point $X_{i+1} \in \partial T$ and an output direction $\mathbf{d}_{i+1}$ in X_{i+1} (see Fig. 11a, b).

To this end, a representation vector in X_i is linearly interpolated from the representation vectors of the vertices S_j of T . The vector $\mathbf{v}_{d_i}$ of the corresponding cross C_i that is closest to the input direction $\mathbf{d}_i$ is selected. The selection of this vector is propagated onto all crosses interpolated over T , thus defining a vector field $Y = f(X)$ over T that is consistent with $\mathbf{d}_i$ (see Fig. 11a). Then we use the Heun's method [29], an easy-to-use variation of a Runge-Kutta method, to integrate $Y = f(X)$ over T , starting from X_i . It consists in computing an intermediate point X'_{i+1} :

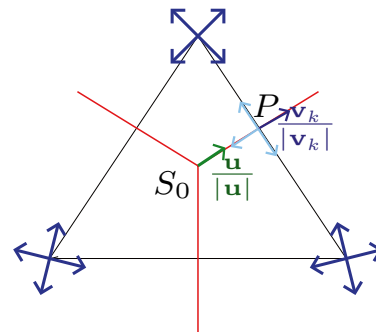


Fig. 10 The intersection of a separatrix (in red) of the singularity S_0 with the triangle around S_0 , at a point P . One of the vector $\mathbf{v}_k$ of the cross at P has the same direction than $\mathbf{u}$, because the separatrix approaches S_0 in a radial direction. Note that the tangent of the separatrix at P is $\mathbf{v}_k$ (color figure online)

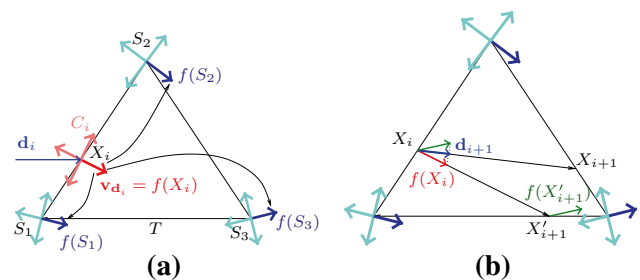


Fig. 11 Example of the integration process over a triangle. In a, given an incoming direction $\mathbf{d}_i$ and an input point X_i , we first determine C_i the cross interpolated in X_i , and then $f(X_i)$ which is the vector of C_i closest to $\mathbf{d}_i$. This selection, extended on all crosses over the triangle, defines a vector field on it. In b, using this vector field and $f(X_i)$ we define an output point X_{i+1} (color figure online)

$$X'_{i+1} = X_i + hf(X_i), \quad (25)$$

and then using $f(X'_{i+1})$ along $f(X_i)$ to compute $\mathbf{d}_{i+1}$:

$$\mathbf{d}_{i+1} = \frac{f(X_i) + f(X'_{i+1})}{2}. \quad (26)$$

Eventually, we compute X_{i+1} :

$$X_{i+1} = X_i + h\mathbf{d}_{i+1}, \quad (27)$$

with h being the integration step (see Fig. 11b). A line can thus be built using segments, by iterating this integration process over the successive encountered triangles.

It is to be noted that, even though the singularities of the representation vector field and the ones of the cross field are identical, their separatrices are totally different. The representation vector field being linked to the cross field by multiplying all the angles the crosses are making with an arbitrary fixed vector (in our case the X axis), streamlines of the representation vector field have absolutely no relevant meaning. On the other hand, the zeroes of both fields having, by definition, an undetermined angle, their locations are unchanged through the transformation from one field to the other.

4.3 Case of a separatrix joining two singularities

In the case of a separatrix going from one singularity to another one, two distinct separatrices are actually computed. Due to numerical round-offs and errors arising during the integration process, those two separatrices are not identical one to the other. To solve this problem, and as each separatrix is more accurate the closer it is from the singularity it spawned from, we compute a linear blending of the two separatrices based on the distance they are from the singularity that spawned them, allowing us to get more accurate results and suppressing a part of the numerical error of the integration scheme. Such a blending can be mathematically expressed easily: given a parametrization $\gamma_1(s), s \in [0, 1]$ of a separatrix going from a singularity S_0 to a singularity S_1 , and a parametrization $\gamma_2(s), s \in [0, 1]$ going from S_1 to S_0 , we compute a blended separatrix γ_b going from S_0 to S_1 and having the following parametrization:

$$\forall s \in [0, 1], \quad \gamma_b(s) = (1 - s) \gamma_1(s) + s \gamma_2(s). \quad (28)$$

4.4 Case of corners featuring small angles

We stated previously that our method provides a domain partitioning into quadrilateral regions. While this remains true in most cases, a problem arises near the corners of the geometry presenting acute angles. Indeed, using our method as such would result in an infinity of streamlines

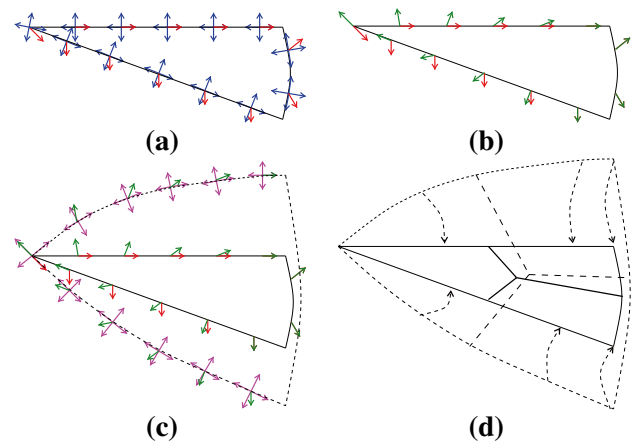


Fig. 12 In **a**, a triangular region at a corner featuring a small angle; the cross field is in *blue* and the corresponding representation vector field is in *red*. In **b**, we show how we change the representation vector at the corner to add π to its angle. Representation vectors on the adjacent edges are then transformed, in *green*, so that the representation vector field is still smooth on the boundary. In **c**, we show in *magenta* the corresponding crosses to the new representation vectors. Using this transformation can be understood as if we considered the angle at the corner to be $\frac{\pi}{2}$ higher than it really is (shown in *dots*), which means that at the resolution, a singularity will be placed inside this region, separating the triangular region into three quadrilaterals, as shown in **d** (color figure online)

ending in such a corner named C_0 . The region featuring such a corner is, therefore, a triangle, and not a quadrilateral (see Fig. 9).

Several methods can be used to handle such problems. We chose to use the opposite of the representation vector at C_0 instead of it and to do a smooth transition between this new representation vector and the ones on the boundary vertices near C_0 . A way to view this modification is to consider that the angle at the corner C_0 is $\frac{\pi}{2}$ higher than it really is and that the crosses smoothly return along the boundary to the prescribed ones (see Fig. 12).

This approach leads to add a three-valent singularity near C_0 , allowing to divide the triangular region into three four-sided regions. This modification consists in either adding one or subtracting one to $S_{\min}$, depending on whether the majority of singularities were of positive index or of negative index, respectively.⁶ In either case, we also added one to $S_{\max}$, which is conveniently the intuitive result.

⁶ This can be understood easily: if there were a majority of negative singularities, at the minimum there were only negative singularities, and one of them could merge with this positive singularity we added and disappear. It is to be noted, though, in this case we can be sure that our algorithm would not reach the minimum, as it would not merge this singularity with another, because it would distort the representation vector field too much. If the majority of singularities were positive, on the contrary, we added one to the minimum.

5 Generation of a quadrilateral mesh

5.1 A quick yet efficient meshing algorithm

Since our algorithm naturally results in a partition composed of four-sided regions, meshing each of these regions can be done efficiently using a bilinear transfinite interpolation. This meshing process is based on the theory of patches and blending developed by Coons [2], and creates a parametrization of the region that coincides with the boundary parametrization. This parametrization leads to an immediate grid meshing process by considering any discretization of the parameter spaces of u and v . The number of irregular vertices of the resulting mesh is equal to the number of singularities of the representation vector field, and, thus, follows the same frame than the singularities of the representation vector field previously defined. It shall be noted that the computational cost of this step, even for millions of elements, is negligible with respect to the cost of the domain partitioning step.

5.2 Improvement on the meshing process using the cross field streamlines

Our method is mainly a partitioning technique before being a meshing technique, albeit the previous algorithm leads to high-quality quadrilateral meshes. It is possible however to improve the generation of the quadrilateral mesh, and hence its overall quality, using our cross field. Indeed, one has to remember that the cross field is designed such that its streamlines are as smooth as possible. To get the best quality for our mesh, it is thus desirable to use these streamlines to define the mesh elements. This can be done by first discretizing the separatrices, taking care of having the same number of points discretizing two opposite curves in each region, ensuring a correspondence for each point on opposite curves. Then, we can build streamlines going from each of those points inside the desired region. Eventually, for each two corresponding points, their streamlines are blended one onto the other, vastly diminishing the numerical error of the integration process. The intersection of the resulting lines can then be used as the vertices of the quadrilateral mesh. While it is more costly to do so, the resulting meshes also show a great improvement in quality, especially along the separatrices of the cross field (see Figs. 13, 14). From what we observed, the computational cost of this step when a mesh of ten thousands elements is desired is roughly equivalent to the one of the domain partitioning (an order of tens of seconds). This cost growth is linear with the amount of elements. For very refined meshes, an intermediate solution could be used by first computing some internal vertices inside each region of the domain with this expensive method, providing a coarse mesh of each region,

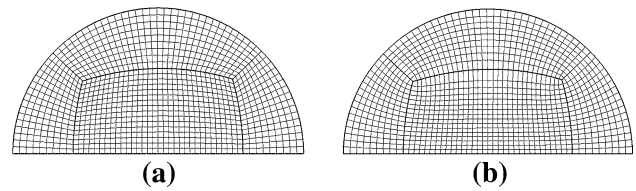


Fig. 13 Comparison between the two meshing methods. In **a**, a mesh obtained through a bilinear transfinite interpolation in each region. In **b**, a mesh obtained through the use of streamlines. The angles of the elements along the separatrices is closer to $\frac{\pi}{2}$ in the second case

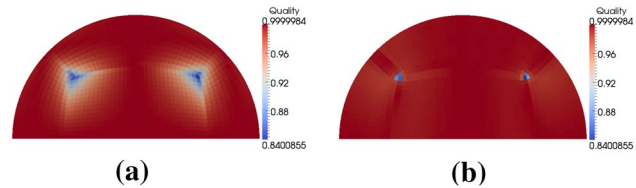


Fig. 14 Comparison between the two meshing methods. In **a**, scaled jacobian quality measurement of the mesh obtained through a bilinear transfinite interpolation in each region. In **b**, scaled jacobian quality measurement of the mesh obtained through the use of streamlines (color figure online)

and then computing the remainder of the vertices locations using the bilinear transfinite technique inside the obtained quadrilaterals. Note that the computation of each streamline are independent one from the other, thus allowing the parallelization of the process should the need arises.

6 Results

In this section, we study some properties of the results we obtain and compare them to the results of an unstructured method through some examples.

6.1 Evolution of the skeleton under refinement of T_h

Aside from the obvious improvements on the various numerical errors, it is interesting to study the effect that the refinement of the background triangular mesh has on our results. We studied the evolution of the skeleton starting from an initial background mesh, that we refined with 4, 16 and 64 times more triangles. The results of this study can be found in Figs. 15, 16, 17 and 18. In Fig. 15, the different background meshes are presented for the initial mesh as well as the mesh refined with four times more triangles, along with a local estimation of the laplacian (and thus, of the error of our solution due to discretization). The local error in Fig. 15b is mostly located in small areas around singularities, and further refinement just diminishes those area sizes. In Figs. 16 and 17, we can see the

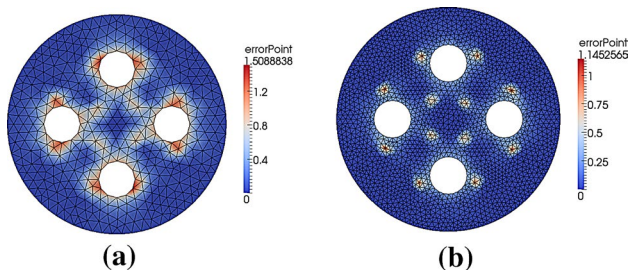


Fig. 15 Background triangular meshes used for the study of the effect of the refinement of the background mesh on the partitioning. A measure of the local error of the cross field is provided alongside. Mesh in **b** has four times more triangles than in **a** (color figure online)

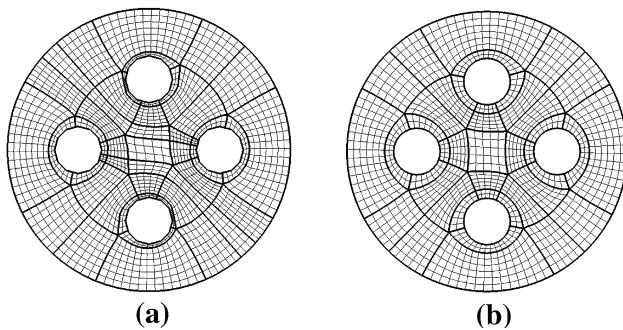


Fig. 16 Study of the effect of the refinement of the background mesh on the partitioning. Background mesh used to obtain the mesh in **a**, **b** are the one in Fig. 15a, b), respectively. In **a**, we can see that a really coarse mesh can lead to numerical trouble in the resolution

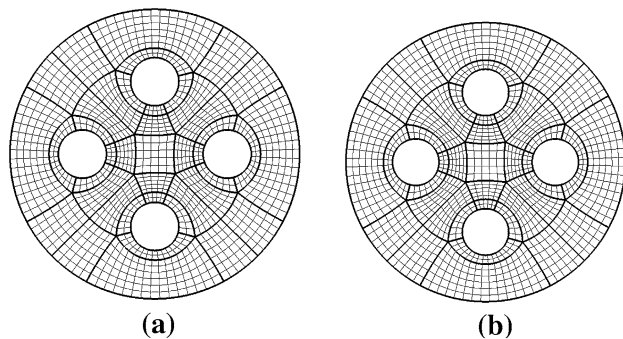


Fig. 17 Study of the effect of the refinement of the background mesh on the partitioning. Background mesh used to obtain the mesh in **a** and **b** have, respectively, 16 and 64 more triangles than the one in Fig. 15a. In **a**, **b** and Fig. 16b, we can see that once the background mesh is fine enough, the obtained results are the same, give or take the numerical round-offs in the resolution

partitioning and the mesh obtained in each case. Except for the Fig. 16a, where numerical errors are predominant, we can observe that the obtained partitioning is identical in each case. In fact, as can be seen on Fig. 18b, the mesh

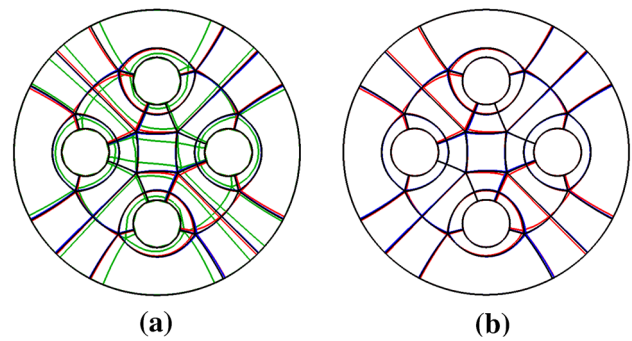


Fig. 18 Study of the effect of the refinement of the background mesh on the partitioning. Here all four domain partitioning are presented, in green, red, blue and black, respectively. In **b** only the three latest are presented. We can see that the partitioning converge to a unique solution (color figure online)

density has little to no effect outside of the vicinity of the singularities, while it is primordial inside their vicinity, to insure a precise localization of the singularities and, thus, a low error.

It follows that, to greatly improve both computational cost and the precision of the algorithm, it would be better to determine a first approximation of the field using a coarse background mesh before using the error obtained on this mesh as a size function for the generation of a new background mesh, which will thus be more refined near the singularities.

6.2 Robustness testing when very thin parts appear

For some geometries, singularities can be positionned in such a way that parallel separatrices appear very close one to the other, forming extremely thin parts. In that case, it is very likely that these separatrices will try to reconnect to the same singularities during the integration process. Depending on the order in which the separatrices are integrated, this could easily lead to numerical issues such as parallel separatrices crossing each others and inverting elements. Figure 19 exhibit a geometry where many of these parallel separatrices appear. As can be observed, our approach has been implemented robustly enough so that parallel separatrices never cross and a proper partitioning is obtained. For meshes that are not very refined, the resulting thin parts will be meshed with a single column of quadrilateral elements.

When two singularities are connected by a separatrix, moving one away will make the separatrix split into two parallel ones, as can be seen in Fig. 20. In that case, thin parts can also appear, when two separatrices almost blend into each other. The minimum size of these parts is controlled by a parameter.

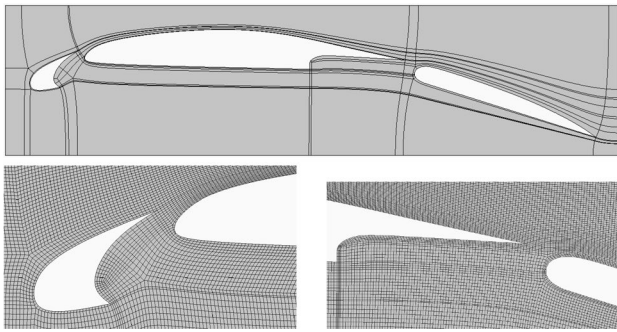


Fig. 19 An aerofoil geometry. Numerous separatrices are very close to each other. Great care has been taken in ensuring each separatrix connects to the best possible singularity, so that separatrices never cross and elements are not inverted. Parts of a resulting mesh are shown below

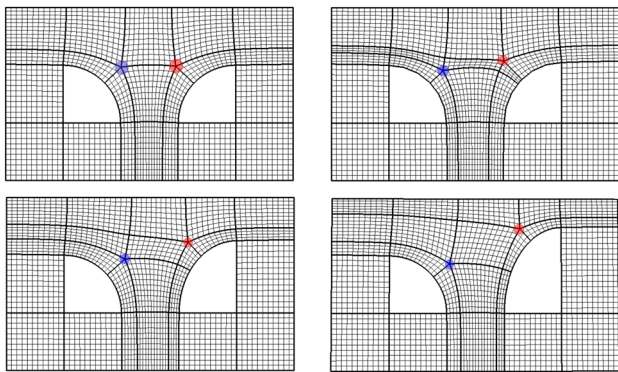


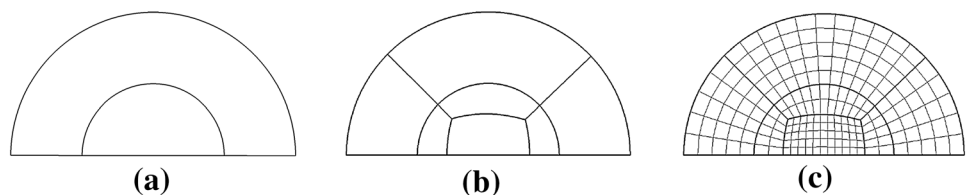
Fig. 20 Singularities tend to be connected by separatrices when they are in symmetrical positions. As they get further away from these positions, the separatrix split into two parallel ones (color figure online)

6.3 Addition of a directional constraint and applications to multidomain meshing

Defining a good quality quadrangulation of a domain while respecting some inner directional constraints is an ongoing issue (see [30], for example). Even though it was not our main purpose, our algorithm fully supports such constraints. Indeed, it is a trivial matter to assign the vertices of the background triangular mesh with a Dirichlet condition representing those constraints, in the same way the boundary directionality is represented (through the representation vectors).

As a consequence, our algorithm can be used to partition and mesh multidomain geometries. While one can

Fig. 21 Partitioning and meshing of a multidomain geometry. In **a**, the geometry. In **b**, the partitioning of the whole geometry. In **c**, the corresponding mesh



partition and mesh each domain separately, problems occur at the interface between the domains and the resulting mesh is usually non-conformal. With our method, the interface can be considered as a boundary, yielding partitionings and meshes of each domain that are conformal at their interface. Figure 21 shows an example of a geometry composed of two domains, with the corresponding partitioning provided by our algorithm as well as the corresponding quadrilateral mesh. Note that the outer domain is partitioned into three regions, which would not be the case if not for the presence of the inner domain.

A more complex example is presented in Fig. 22, where the inside and the outside of a shuttle placed in an ellipsoid are partitioned and discretized together. It should be noted that separatrices emanating from a singularity of a domain either spread through the other domain or join a separatrix from the other domain.

6.4 Quality comparison of our obtained meshes with ones obtained through usual algorithms

While it is not easy to define a quality measurement for a domain partitioning, it is a lot more convenient to define one for quadrilateral meshes. In this section, we compare meshes obtained with our approach with a common advancing front algorithm. Both the minimum angle measure and the scaled jacobian measure are used to compare the results. The two meshes are presented on Fig. 23; it is to be noted that both meshes have about 9,000 quadrilaterals. On Fig. 24, we present the minimum angle measure as color maps for each mesh. On Fig. 25, we do the same for the scaled jacobian measure. Eventually, on Fig. 26, we present histograms of those results.

For the minimum angle quality measure, the unstructured algorithm has a worst quality of 36.8° and a mean quality of 77.3° . With our algorithm, the worst quality is of 66.4° whereas the mean quality is of 86.4° .

For the Jacobian quality measure, the unstructured algorithm has a worst quality of 0.5 and a mean quality of 0.95. With our algorithm, the worst quality is of 0.92 and the mean quality of 0.996.

6.5 Additional examples

In Figs. 27 and 28, we show some other results obtained through our algorithm. In Fig. 27a, the domain consists

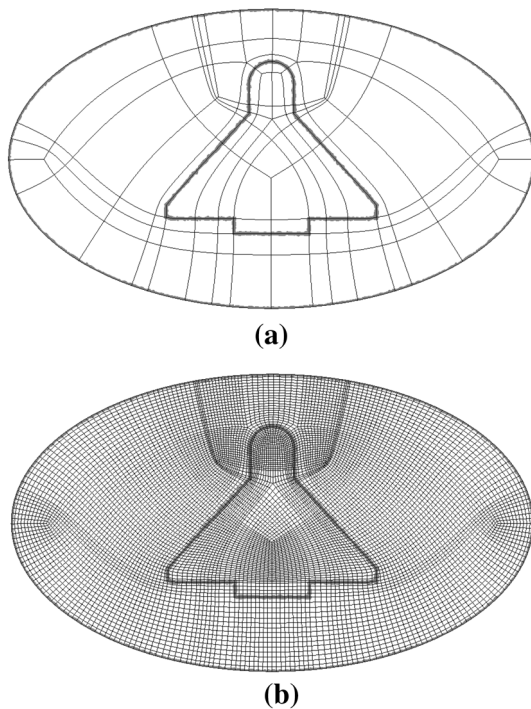


Fig. 22 Two domains sharing a curve are discretized separately: domain partitioning in **a** and quadrilateral meshes in **b**

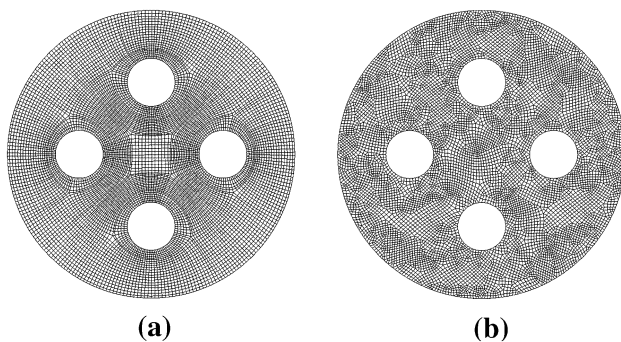
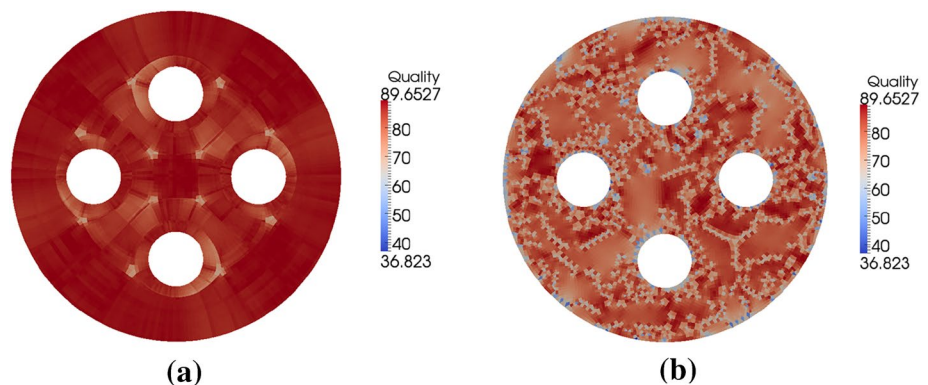


Fig. 23 In **a**, a mesh obtained through our algorithm. In **b**, the mesh produced on the same geometry by a totally unstructured algorithm

Fig. 24 Study of the minimum angle quality measure on our mesh (in **a**) and on the unstructured mesh (in **b**) as a color map (color figure online)



in four circle halves around a square. As for the previous examples, the symmetry is perfectly preserved through the process. In Fig. 27b, the domain is really close, although the portion of circles on the boundary are now less than a half, introducing changes in the location of singularities. In Fig. 27c, a hole is inserted in the geometry, leaving the domain with no axis of symmetry and introducing new singularities. The quality of the elements remains excellent.

In Fig. 28a, we obtain a mesh with no singularity for a complex geometry, which no current method would automatically produce, as far as we know. As a consequence, each of the obtained quadrilateral is perfect, save for the error due to the discretization size. In Fig. 28b, the mesh obtained for a mechanical piece is presented.

7 Conclusion

In this paper, we presented a novel approach to domain partitioning, which can be used as an efficient block-structured quadrilateral mesh generation algorithm. This fully automatic algorithm uses the solution of a PDE to extend the information contained by normals and tangents at the boundary inside the domain. It generates a partition of the domain into four-sided regions with curvilinear edges that can be easily meshed into quadrilateral elements. The number of singular vertices in the resulting mesh is minimal or close to minimal. We also provided a method to generate such a mesh using our solution field, providing a resulting mesh with near perfect quality for all elements not adjacent to one of the few singular vertices. Being an algebraic method at its core, our method provides results that are independent of the initial orientation of the domain and that respects its geometrical symmetries. An added benefit of our method is that the main computational cost is due to the domain partitioning. As a result, the additional cost for having a more refined mesh is very low, making our method quicker than most

Fig. 25 Study of the Jacobian quality measure on our mesh (in **a**) and on the unstructured mesh (in **b**) as a color map (color figure online)

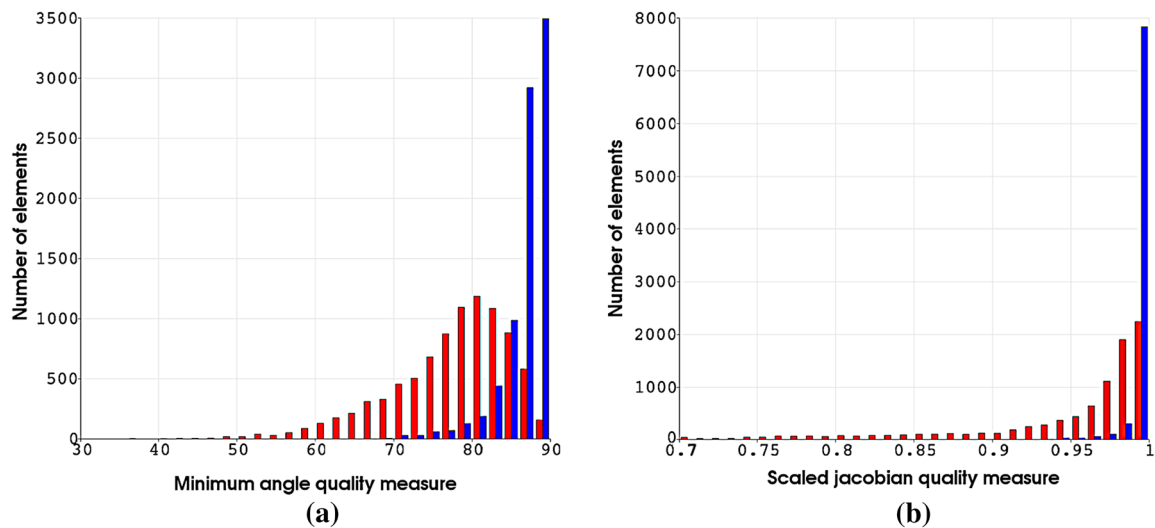
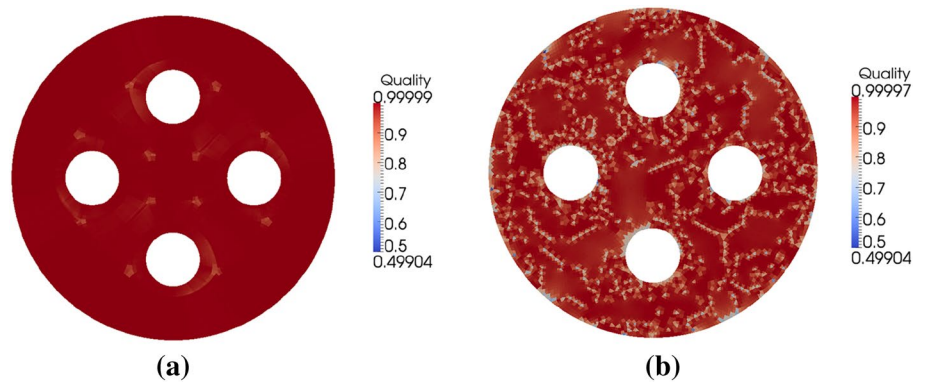


Fig. 26 Histograms representing the repartition of elements by quality, the unstructured mesh being represented in *red* and our mesh in *blue*. In **a** the *histogram* represents the repartition for the minimum

angle quality, and in **b** it represents the repartition for the Jacobian quality (color figure online)

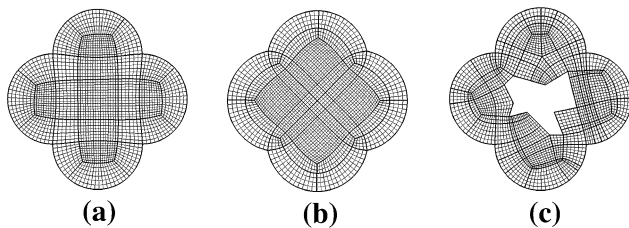


Fig. 27 Additional examples

other methods for refined meshes. Although the meshing part of the process is currently unable to deal with varying prescribed element sizes, future work is scheduled to improve this using known methods to refine the mesh inside desired regions. Other approaches to solve this issue are already studied [31]. Eventually, it should

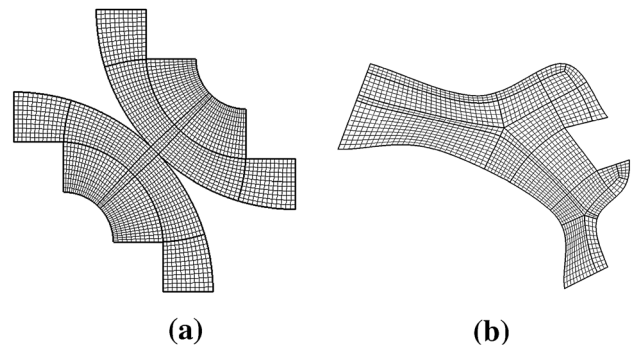


Fig. 28 Additional examples

be noted that the partitions obtained through our method could help with the generation of higher order meshes by providing meshes that follow the flow field before elements are curved.

References

1. Cook RD, Malkus DS, Plesha ME (1989) Concepts and applications of finite element analysis, 3rd edn. Wiley, New York
2. Faux ID, Pratt MJ (1979) Computational geometry for design and manufacture. Ellis Horwood, Chichester
3. Whiteley M, White D, Benzley S, Blacker T (1994) Two and three-quarter dimensional meshing facilitators. *Eng Comput* 12:144–145
4. Schneiders R (1996) A grid-based algorithm for the generation of hexahedral element meshes. *Eng Comput* 12:168–177
5. Staten ML, Owen SJ, Blacker T (2005) Unconstrained paving and plastering: a new idea for all hexahedral mesh generation. *Proc Int Mesh Roundtable* 14:399–416
6. Borouchaki H, Frey PJ (1998) Adaptive triangular-quadrilateral mesh generation. *Int J Numer Methods Eng* 41:915–934
7. Owen SJ, Staten ML, Canann SA, Saigal S (1999) Q-morph: an indirect approach to advancing front quad meshing. *Int J Numer Methods Eng* 44:1317–1340
8. Lévy B, Liu Y (2010) Lp centroidal voronoi tessellation and its applications. *ACM Trans Graph* 29. doi:[10.1145/1778765.1778856](https://doi.org/10.1145/1778765.1778856)
9. Remacle JF, Lambrechts J, Seny B, Marchandise E, Johnen A, Geuzaine C (2012) Blossom-quad: a non-uniform quadrilateral mesh generator using a minimum cost perfect matching algorithm. *Int J Numer Methods Eng* 89:1102–1119
10. Lin W, Tang Y, Zhao C, Liu X, Zhu G, Jiang F (2011) An algorithm for automatic 2D finite element mesh generation with line constraints. *Comput Aided Des* 43:1803–1813
11. Park C, Noh JS, Jang IS, Kang JM (2007) A new automated scheme of quadrilateral mesh generation for randomly distributed line constraints. *Comput Aided Des* 39:258–267
12. Tam TKH, Armstrong CG (1991) 2D finite element mesh generation by medial axis subdivision. *Adv Eng Softw Workstn* 13:313–324
13. Bommers D, Lévy B, Pietroni N, Puppo E, Silva C, Tarini M (2013) Zorin D quad-mesh generation and processing: a survey. *Comput Graph Forum* 32:51–76
14. Durst F, Schfer M (1996) A parallel block-structured multigrid method for the prediction of incompressible flows. *Int J Numer Methods Fluids* 22:549–565
15. Lions PL (1990) On the Schwarz alternating method III: a variant for nonoverlapping subdomains. In: Third international symposium on domain decomposition methods for partial differential equations, Philadelphia
16. Romé C, Glockner S, Caltagirone JP (2007) Resolution of the Navier–Stokes equations on block-structured meshes. *Int J Numer Methods Fluids* 54:1239–1268
17. Wulf WA, McKee SA (1995) Hitting the memory wall: implications of the obvious. *Computer architecture and news*, Association for Computing
18. Rude U (1997) Iterative algorithms on high performance architectures. In: *Proceedings of Europar'97*, Lecture notes in computer science, Springer, pp 57–71
19. Ray N, Vallet B, Li W, Lévy B (2008) N-Symmetry direction field design. *ACM Trans Graph* 27. doi:[10.1145/1356682.1356683](https://doi.org/10.1145/1356682.1356683)
20. Palacios J, Zhang E (2007) Rotational symmetry field design on surfaces. *ACM Trans Graph* 26. doi:[10.1145/1239451.1239506](https://doi.org/10.1145/1239451.1239506)
21. Strang G, Fix G (1973) An analysis of the finite element method. Wellesley-Cambridge Press
22. Alouges F (1997) A new algorithm for computing liquid crystal stable configurations: the harmonic mapping case. *SIAM J Numer Anal* 34:1708–1720
23. Saad Y (2003) Iterative methods for sparse linear systems. SIAM, Philadelphia
24. Bramble JH (1981) The Lagrange multiplier method for Dirichlet's problem. *Math Comput* 37:1–11
25. Tricoche X (2002) Vector and tensor fields topology simplification, tracking and visualization. Ph.D thesis
26. Tricoche X, Scheuermann G, Hagen H (2001) Continuous topology simplification of planar vector fields. In: *Proceedings of IEEE visualization*, pp 159–166
27. Helman JL, Hesselink L (1991) Visualizing vector field topology in fluid flows. *IEEE Comput Graph Appl* 11:36–46
28. White DR (1995) Automated hexahedral mesh generation by virtual decomposition. In: *Proceedings, 4th international meshing roundtable*, pp 165–176
29. Ascher UM, Petzold LR (1998) Computer methods for ordinary differential equations and differential-algebraic equations. SIAM, Philadelphia
30. Liu Y, Xing HL, Guan Z (2011) An indirect approach for automatic generation of quadrilateral meshes with arbitrary line constraints. *Int J Numer Methods Eng* 87:906–922
31. Fogg H, Armstrong C, Robinson TT (2013) MultiBlock decomposition using cross-fields. In: *ADMOS 2013—international conference on adaptive modeling and simulation*